

FPGA-Based Glass Bridge Survival Game

- Digital Logic Design Laboratory Project Report

1. OBJECTIVE

- Implement a memory-based survival game using Finite State Machine (FSM) on FPGA
- Use Linear Feedback Shift Register (LFSR) for random safe path generation
- Validate player choices using comparator logic with switch inputs
- Display game progress using LEDs and 7-segment displays
- Demonstrate practical digital logic design concepts on hardware

2. INTRODUCTION

2.1 Background

The Glass Bridge Survival Game challenges players to cross a virtual bridge by memorizing and selecting safe glass panels. The game displays a safe path briefly, then players must recall and navigate the pattern correctly to advance.

2.2 Game Mechanics

- **4 Progressive Stages:** Increasing difficulty from 2 to 12 columns
- **Memory Challenge:** Pattern shown once, then hidden during gameplay
- **Binary Outcome:** Correct choice advances, wrong choice eliminates player

3. PROJECT OVERVIEW

The system generates a random bridge pattern using LFSR, displays it to the player for memorization, then validates the player's navigation choices in real-time. The game features four difficulty stages with increasing complexity and faster display speeds.

Key Features:

- Multi-stage progression (2→3→7→12 columns)
- Real-time visual feedback via LEDs
- 7-segment display showing row position and game state
- Reliable button input handling with debouncing

4. HARDWARE REQUIREMENTS

Component	Purpose
Basys 3 FPGA Board	Main platform (Xilinx Artix-7)
100 MHz Clock	System timing
4-Digit 7-Segment Display	Shows stage, row number, game status
16 LEDs	Pattern display and feedback
16 Switches (SW0-SW15)	Glass panel selection
3 Push Buttons	Start game, advance row, reset
USB Cable	Programming and power

5. SOFTWARE REQUIREMENTS

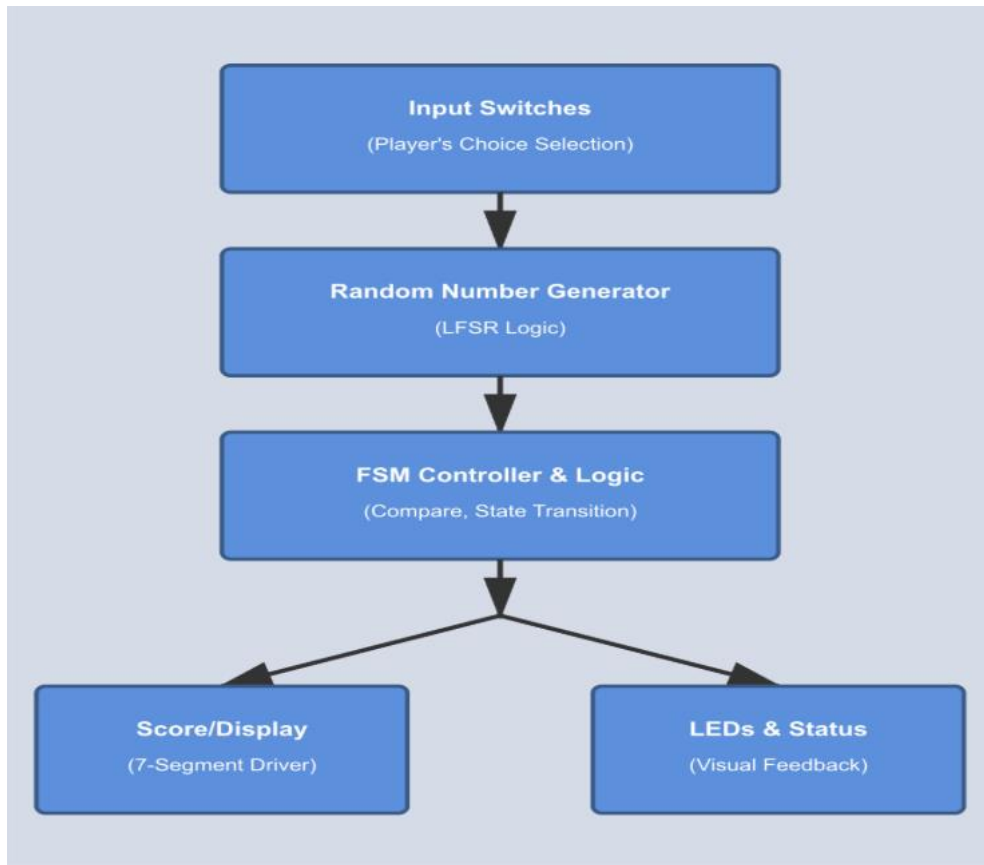
Software	Purpose
Xilinx Vivado Design Suite	HDL design, simulation, synthesis
Verilog HDL	Hardware description language
Vivado Simulator	Functional verification
Hardware Manager	FPGA programming

6. SYSTEM ARCHITECTURE

6.1 Main Modules

Module	Function
Input Synchronization	Prevents metastability from button inputs
Button Debounce	Eliminates mechanical bounce (5ms filter)
LFSR Generator	Creates random patterns (32-bit)
FSM Controller	Manages 7 game states
Bridge Storage	15×12 matrix storing safe/unsafe panels
Comparator	Validates player choice
Display Multiplexer	Controls 4-digit 7-segment display
LED Driver	Shows patterns and feedback

6.2 Block Diagram



7. HDL CODE - KEY SECTIONS

7.1. Module Declaration & Inputs/Outputs

```
module glass_bridge(  
    input clk,           // 100MHz clock  
    input rst,           // Reset button  
    input btn_start,     // Center button to start  
    input btn_step,      // Up button to move to next row  
    input [15:0] sw,     // Switches for glass selection  
    output reg [15:0] led, // LEDs to show glass pattern  
    output reg [6:0] seg, // 7-segment display  
    output reg [3:0] an   // 7-segment anodes  
);
```

This is a game inspired by "Squid Game" where you must cross a bridge by choosing safe glass panels. The module takes clock input, reset, two buttons, and 16 switches, and outputs to 16 LEDs and a 4-digit 7-segment display.

7.2. LFSR Random Number Generator

```
wire lfsr_feedback = lfsr[31] ^ lfsr[21] ^ lfsr[1] ^ lfsr[0];  
  
always @(posedge clk) begin  
    if (rst_sync2)  
        lfsr <= 32'hACE1FACE;  
    else  
        lfsr <= {lfsr[30:0], lfsr_feedback};  
end
```

Pseudo-random generation: Uses XOR feedback from specific bits (31, 21, 1, 0) to create a maximal-length sequence. Shifts left each clock cycle, inserting the feedback bit at position 0. This generates the random bridge pattern.

7.3. Choice Validation Logic

```
always @(*) begin
    choice_valid = 0;
    case (num_cols)
        2: begin
            if ((sw[0] && bridge[current_row][0]) ||
                (sw[1] && bridge[current_row][1]))
                choice_valid = 1;
            end
        // ...
    endcase
end
```

Combinational logic that checks if any activated switch corresponds to a safe panel (value 1) in the current row. If the switch position matches a safe panel, choice_valid becomes 1.

7.4. SHOW_PATTERN State

```
SHOW_PATTERN: begin
    timer <= timer + 1;

    if (timer < display_time) begin
        // Show current row pattern on LEDs
        case (num_cols)
            2: led <= {{14{1'b0}}, bridge[display_row][1:0]};
            // ...
        endcase
    end else if (timer < display_time + off_time) begin
        led <= 16'h0000; // Blank period
    end else begin
        // Move to next row or finish showing
    end
end
```

Pattern memorization phase:

- Displays each row's pattern on the LEDs for display_time
- Turns LEDs off for off_time between rows

- Cycles through all rows so player can memorize the safe path
- Right-aligns the pattern (uses lower bits of LED array)

7.5. 7-Segment Display Multiplexing

```
always @(posedge clk) begin
    if (digit_counter < 17'd100000) begin
        digit_counter <= digit_counter + 1;
    end else begin
        digit_counter <= 0;
        digit_select <= digit_select + 1; // Cycle through digits
    end
end
end
```

Time-multiplexing: The 4-digit display shares the same 7 segment lines. This code rapidly cycles through each digit (switching every 1ms at 100MHz) faster than the eye can see, creating the illusion all digits are on simultaneously.

7.6. 7-Segment Display Content & Decoding

```
always @(*) begin
    case (state)
        IDLE: begin
            // Show "000X" where X is stage number
            case (digit_select)
                2'd0: display_digit = stage;
                // ...
            endcase
        end
    end
end
```

```

        SHOW_PATTERN, WAIT_INPUT, CHECK_CHOICE: begin
            // Show "XXYY" (current row / total rows)
            case (digit_select)
                2'd0: display_digit = total_rows % 10;
                2'd1: display_digit = total_rows / 10;
                2'd2: display_digit = (current_row + 1) % 10;
                2'd3: display_digit = (current_row + 1) / 10;
            endcase
        end
        // ...
    endcase

    // 7-segment decoder
    case (display_digit)
        4'd0: seg = 7'b1000000; // Pattern for "0"
        4'd1: seg = 7'b1111001; // Pattern for "1"
        // ...
    endcase
end

```

Display shows different information per state:

- **IDLE:** "0001" to "0004" (stage number)
- **During game:** "0310" means row 3 of 10
- **WIN:** "9999"
- **LOSE:** "0000"

The 7-segment decoder converts binary numbers (0-9) to the specific segment patterns (active-low encoding for common anode displays).

7.7. WIN State

```
WIN: begin
    timer <= timer + 1;
    if (timer[24]) // Flash all LEDs
        led <= 16'hFFFF;
    else
        led <= 16'h0000;

    if (btn_start_pressed) begin
        if (stage < 4)
            stage <= stage + 1; // Next stage
        else
            stage <= 1; // Loop back to stage 1
        state <= IDLE;
    end
end
```

Victory animation with all LEDs flashing. Press start to advance to next stage or restart at stage 1 after beating stage 4.

7.8. LOSE State

```
LOSE: begin
    timer <= timer + 1;
    if (timer[24])
        led <= 16'hAAAA; // Alternating pattern
    else
        led <= 16'h5555;

    if (btn_start_pressed) begin
        stage <= 1; // Reset to stage 1
        state <= IDLE;
    end
end
```

Defeat animation with alternating LED pattern (0xAAAA = 1010..., 0x5555 = 0101...). Press start to restart from stage 1.

8. WORKING PRINCIPLE

8.1 Game Flow

1. Initialization (IDLE State)

- Display shows stage number (0001-0004)
- Wait for START button press

2. Pattern Generation (GEN_BRIDGE State)

- LFSR generates random safe panels
- Creates bridge matrix (15 rows × up to 12 columns)
- Each row guaranteed to have at least one safe panel

3. Pattern Display (SHOW_PATTERN State)

- Shows each row's pattern on LEDs
- Display time: 1s (Stage 1) down to 0.3s (Stage 4)
- Brief pause between rows
- Player must memorize the pattern

4. Gameplay (WAIT_INPUT State)

- Single LED blinks showing current row
- 7-segment shows "row X of Y"
- Player flips switches to select panel
- Press STEP button to confirm

5. Validation (CHECK_CHOICE State)

- Compares switch position with safe panels
- Shows feedback for 0.5 seconds:
 - All LEDs ON = Correct
 - All LEDs OFF = Wrong

6. Outcomes

- **Correct:** Advance to next row
- **Wrong:** Game over (LOSE state)
- **Complete all rows:** Win stage (WIN state)

8.2 Stage Progression

Stage	Rows	Columns	Display Time	Difficulty
1	10	2	1.0s	Easy
2	10	3	0.75s	Medium
3	15	7	0.5s	Hard
4	15	12	0.3s	Expert

9. FUTURE ENHANCEMENTS

1. **Multiplayer Mode:** Add 2-4 player support with turn-based gameplay
2. **Difficulty Adjustment:** Auto-adjust speed based on player performance
3. **Timer System:** Add countdown timer for speed challenges
4. **VGA Output:** Display graphical bridge on monitor
5. **Sound Effects:** Add audio feedback using PWM

10. DISCUSSION

Throughout the development of the FPGA-Based Glass Bridge Survival Game, we faced several practical challenges that helped shape the final design. Calibrating the timing of LEDs and displays required careful adjustment to keep the game difficulty balanced and consistent. Generating random yet fair bridge patterns using an LFSR also demanded experimentation to avoid predictable gameplay. We also had to consider overall user experience—ensuring that button presses, stage progression, and visual feedback felt smooth and intuitive. From these challenges, we gained valuable hands-on learning. We became more confident in implementing FSMs on actual hardware, handling clock synchronization issues, and applying proper debouncing techniques. The project also strengthened our understanding of memory use inside FPGA devices and highlighted the importance of clear, user-focused interface design. Overall, this work demonstrated how game logic, real-time interaction, random number generation, and multi-module integration come together in a real digital system.

11. CONCLUSION

In conclusion, the FPGA-Based Glass Bridge Survival Game successfully brought together multiple concepts from Digital Logic Design into one functional hardware project. The system uses a 7-state FSM to control the entire game flow, an LFSR for random pattern generation, and well-timed operations synchronized to the main clock. Input from switches and buttons, along with LED and display outputs, forms a simple yet effective user interface that makes the game easy to understand and play. With proper synchronization and debouncing, the game runs reliably on the FPGA board. This project gave us meaningful experience in HDL coding, timing control, and hardware-based system design. Ultimately, we produced a fully working and enjoyable game that clearly demonstrates how core digital logic principles can be applied creatively in real hardware.

14. REFERENCES

1. Lab Slides
2. Xilinx Basys 3 FPGA Reference Manual
3. Verilog HDL Programming Guide